

P0-6 Runbook — VPS hardening & pipeline runtime

SECTION 06-operations-it

TYPE runbook

STATUS **active**

OWNER Founder

UPDATED 2026-06-16

Companion to `setup-vps.md` (which covers the *why*). This file is the *exact how* — copy-paste commands, in a safe order, with verification checks and rollback notes.

Server: Hostinger KVM 2 (2 vCPU / 8 GB / 100 GB NVMe) **IP / hostname:** `72.62.20.20` / `srv1209075.hstgr.cloud` **OS:** Ubuntu 24.04 LTS · **Admin user:** `adel` (sudo) **Role:** public-data-only pipeline box — RAG indexing, eval harness, staging. Never production, never personal data. (See `setup-vps.md` → "Data boundary (PDPL)".)

! Golden rules — read before you start

1. **Never close your working SSH session** until a *fresh* session is proven to work with the new config. Steps 4 and 6 can lock you out if rushed.
2. **Keep two terminal windows ready** on the Mac for Steps 4 and 6 — one is your safety line, the other is the test.
3. **If you get locked out:** Hostinger hPanel → VPS → *Browser terminal* (or VNC console) gives you root access that does not depend on SSH. Rollback commands are at the bottom of this file.
4. Run the steps **in order**. The OS update comes first; SSH hardening comes last (it is the most dangerous, and by then key login is already proven).

Step 1 — Verify access (baseline)

From the Mac's Terminal:

```
ssh adel@72.62.20.20
```

You should land in a shell **without being asked for a password** (the SSH key did it). If it asked for a password, stop — the key is not installed and Step 6 would lock you out. Then, on the VPS:

```
whoami                # → adel
sudo whoami           # → root   (may prompt for adel's sudo password — fine)
lsb_release -d        # → Ubuntu 24.04.x LTS
uname -r              # kernel version
cat ~/.ssh/authorized_keys # must show your public key — confirms key login survives
Step 6
```

Checkpoint: `adel`, `root`, `Ubuntu 24.04`, and a non-empty `authorized_keys`. Do not proceed past Step 6 unless `authorized_keys` shows your key.

Step 2 — Update the OS

```
sudo apt update
sudo apt full-upgrade -y
sudo apt autoremove --purge -y
```

If the kernel or `libc` was upgraded, reboot and reconnect:

```
sudo reboot
# wait ~30 s, then from the Mac:
ssh adel@72.62.20.20
```

Enable automatic security updates (non-interactive):

```
sudo apt install -y unattended-upgrades
echo 'APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";' | sudo tee /etc/apt/apt.conf.d/20auto-upgrades
```

Checkpoint: `apt` finishes with no held/broken packages.

Step 3 — Install firewall, fail2ban, and the pipeline runtime

Base tools (one line):

```
sudo apt install -y ufw fail2ban git curl ca-certificates build-essential software-properties-common
```

Python 3.11 — Ubuntu 24.04 ships 3.12; the RAG pipeline targets 3.11, so add it via the deadsnakes PPA (3.12 stays as the system default; we just add 3.11 alongside):

```
sudo add-apt-repository -y ppa:deadsnakes/ppa
sudo apt update
sudo apt install -y python3.11 python3.11-venv python3.11-dev
python3.11 --version          # → Python 3.11.x
```

Node.js 20 — via NodeSource:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node --version          # → v20.x
npm --version
```

git (installed above):

```
git --version
```

Checkpoint: `python3.11`, `node v20.x`, `npm`, `git` all report versions.

Step 4 — Configure the firewall (ufw)

⚠ Allow SSH *before* enabling ufw, or you lock yourself out. Run these in order:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH          # opens 22/tcp via the app profile
sudo ufw enable                 # answer 'y' at the prompt
sudo ufw status verbose
```

`ufw enable` does **not** drop your current (established) connection. Still, verify: **open a second Terminal window** on the Mac and run `ssh adel@72.62.20.20`. If it connects, ufw is safe. Keep both windows open into Step 6.

Checkpoint: `ufw status verbose` shows `Status: active` and an `OpenSSH ALLOW` rule; a fresh SSH session connects.

Step 5 — Configure fail2ban

Ubuntu 24.04 logs auth to journald (no `/var/log/auth.log` by default), so the jail must use the **systemd** backend:

```

sudo tee /etc/fail2ban/jail.local > /dev/null <<'EOF'
[DEFAULT]
bantime = 1h
findtime = 10m
maxretry = 5
backend = systemd

[sshd]
enabled = true
EOF

sudo systemctl enable --now fail2ban
sudo systemctl restart fail2ban
sudo systemctl is-active fail2ban      # → active
sudo fail2ban-client status            # → Jail list: sshd
sudo fail2ban-client status sshd       # → shows the sshd jail detail

```

Checkpoint: `fail2ban` is `active` and `fail2ban-client status` lists `sshd`.

Step 6 — Harden SSH (root login + password auth OFF)

 **Most dangerous step. Keep your current session open. Have a second window ready.**

First, see what is already there (cloud images sometimes ship a drop-in that re-enables password auth):

```

ls -la /etc/ssh/sshd_config.d/
sudo grep -RinE 'passwordauthentication|permitrootlogin' /etc/ssh/sshd_config
/etc/ssh/sshd_config.d/

```

Create our drop-in. It is named `01-` so it sorts **first** — in `sshd_config`, the first value wins, so this overrides any later `50-cloud-init.conf`:

```

sudo tee /etc/ssh/sshd_config.d/01-flygaca-hardening.conf > /dev/null <<'EOF'
# Fly GACA VPS hardening — P0-6
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
PermitEmptyPasswords no
MaxAuthTries 3
EOF

```

Test the syntax, then check the **effective merged** config — this is the real test:

```

sudo sshd -t && echo "syntax OK"
sudo sshd -T | grep -Ei
'permitrootlogin|passwordauthentication|kbdinteractive|pubkeyauthentication'

```

Expected output:

```
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no
kbdinteractiveauthentication no
```

If `passwordauthentication` still shows `yes`, a conflicting drop-in is being read first — open the offending file from the `grep` above and comment out its line, then re-check. **Do not restart ssh until `sshd -T` shows the four expected values.**

Apply:

```
sudo systemctl restart ssh
```

Now verify before closing anything. In the *second* Terminal window:

```
ssh adel@72.62.20.20      # must log in with the key
ssh root@72.62.20.20     # must be REFUSED
```

Only once the new `adel` session works **and** root is refused, close the old window.

Checkpoint: fresh `adel` key login works; `root` login refused; `sshd -T` shows the four expected values.

Step 7 — Confirm and record the region

```
curl -s ipinfo.io
```

Note the `city` / `region` / `country` / `org` fields. Cross-check against Hostinger hPanel → VPS → *Overview* → datacenter location (the panel is authoritative; `ipinfo.io` is geolocation and can be approximate). A Europe location (Germany or France) is expected and acceptable — the VPS holds public data only.

Record the confirmed region in `../phase0.md` (Step 8).

Step 8 — Record the result

Update `../phase0.md` → **P0-6**: tick the completed checkboxes, set the status to `Done`, and fill in the region and installed runtime versions. (Done in the Cowork workspace alongside this runbook.)

Final state — what "done" looks like

Item	Expected
OS	Ubuntu 24.04 LTS, fully updated, auto security updates on
SSH	key-only; root login disabled; password auth disabled
Firewall	<code>ufw</code> active — only OpenSSH (22) inbound
Brute-force protection	<code>fail2ban</code> active, <code>sshd</code> jail enabled
Python	3.11.x available (<code>python3.11</code>) alongside system 3.12
Node.js	v20.x + npm
git	installed
Region	confirmed and recorded in <code>phase0.md</code>

Rollback — if something goes wrong

Use the **Hostinger hPanel → VPS → Browser terminal / VNC console** for all of these. It is a root console that does not depend on SSH or the firewall.

Locked out after Step 6 (SSH hardening):

```
rm /etc/ssh/sshd_config.d/01-flygaca-hardening.conf
systemctl restart ssh
```

Locked out after Step 4 (firewall):

```
ufw disable
```

fail2ban banned your own IP:

```
fail2ban-client set sshd unbanip <your-ip>
```

(Find your IP with `curl -s ifconfig.me` from the Mac.)

Part of the Fly GACA Phase 0 setup walkthroughs. Not legal advice; PDPL data boundary per `setup-vps.md`.